8. Architecture Overview

8.1 High-Level System Architecture

The system follows a clean **three-tier architecture**:

1. **Presentation Tier:** React SPA designed in Stitch. Handles routing, state management, UI rendering, and browser Geolocation API calls.
2. **Application Tier:** FastAPI modular monolith. Handles authentication, authorization, business logic, data validation, and email dispatch.
3. **Data Tier:** MongoDB database. Stores users, events, registrations, and notifications.

Communication is exclusively via REST/JSON. The frontend sends JWTs in the `Authorization: Bearer <token>` header for protected endpoints.

8.2 Backend Module Architecture

8.3 Frontend–Backend Boundary

Responsibility	React (Frontend)	FastAPI (Backend)
UI rendering	✅	❌
Client-side routing	✅	❌
Form validation (UX)	✅ Basic	❌
Data validation (security)	❌	✅ Pydantic
Business logic	❌	✅
Authentication state	✅ Store/send JWT	✅ Issue/verify JWT
Authorization	❌	✅ Enforce roles
Database access	❌	✅ Motor
Email delivery	❌	✅
Browser geolocation	✅ Collect coordinates	✅ Store & query

8.4 Recommended Project Structure

```
EventAdda/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── core/
│   │   │   ├── __init__.py
│   │   │   ├── database.py
│   │   │   ├── security.py
│   │   │   └── dependencies.py
│   │   ├── auth/
│   │   │   ├── __init__.py
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   └── service.py
│   │   ├── events/
│   │   │   ├── __init__.py
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   └── recommendations.py
│   │   ├── registrations/
│   │   │   ├── __init__.py
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   └── service.py
│   │   ├── notifications/
│   │   │   ├── __init__.py
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   └── email.py
│   │   ├── onboarding/
│   │   │   ├── __init__.py
│   │   │   ├── routes.py
│   │   │   └── schemas.py
│   │   └── profile/
│   │       ├── __init__.py
│   │       ├── routes.py
│   │       └── schemas.py
│   ├── scripts/
│   │   ├── seed_data.py
```

```
│   │   └── cities.py
│   ├── tests/
│   │   ├── __init__.py
│   │   ├── conftest.py
│   │   ├── test_auth.py
│   │   ├── test_events.py
│   │   ├── test_registrations.py
│   │   └── test_notifications.py
│   ├── .env.example
│   ├── requirements.txt
│   └── README.md
├── frontend/
│   └── ... (React app, managed by frontend team)
├── docs/
│   ├── api_contract.md
│   ├── decisions.md
│   └── mock_responses/
│       ├── events_list.json
│       ├── recommendations.json
│       └── user_profile.json
├── .gitignore
└── README.md
```

File Responsibilities

File / Directory	Purpose	Must NOT Contain
`main.py`	FastAPI app instance, CORS config, router registration, startup/shutdown events	Business logic, database queries
`config.py`	Pydantic `Settings` class reading from `.env`	Hardcoded secrets, business logic
`core/database.py`	Motor async client, `get_database()` dependency	Route definitions, schemas
`core/security.py`	`hash_password()`, `verify_password()`, `create_access_token()`, `decode_access_token()`	Database calls, route definitions
`core/dependencies.py`	`get_current_user`, `require_role()`, `require_organizer_owner()` FastAPI dependencies	Business logic beyond auth checks
`*/routes.py`	FastAPI router with endpoint definitions; calls service functions	Direct database queries, complex logic
`*/schemas.py`	Pydantic request/response models	Business logic, database access
`*/service.py`	Business logic functions; receives database handle, returns data	FastAPI `Request` objects, route decorators
`events/recommendations.py`	Recommendation scoring algorithm	Route definitions, authentication logic
`notifications/email.py`	Email composition and sending (SMTP or console)	Database access, notification storage
`scripts/seed_data.py`	Seed dummy events, users, registrations for demo	Production code; should be run manually
`scripts/cities.py`	Static list of ~50 Indian cities with coordinates	—
`tests/conftest.py`	pytest fixtures: test client, test database, test users	Production code
`docs/api_contract.md`	API specification shared with frontend team	Implementation details
`docs/mock_responses/`	JSON files matching API response schemas for frontend development	—

9. MongoDB Data Model

9.1 Collections Overview

9.2 `users` Collection

Field Descriptions

Field	Type	Required	Default	Description
`_id`	ObjectId	Auto	—	MongoDB primary key
`email`	String	✅	—	User's email address (unique, case-insensitive)
`phone`	String	✅	—	User's phone number (unique, E.164 or 10-digit)
`full_name`	String	✅	—	User's display name
`password_hash`	String	✅	—	bcrypt hash of the password
`role`	String	✅	`"participant"`	Enum: `"participant"`, `"organizer"`, `"both"`
`interests`	Array[String]	❌	`[]`	Selected interest categories
`location`	Object	❌	`null`	Embedded: `{ city, locality?, coordinates? }`
`location.city`	String	If location set	—	City name (e.g., "Pune")
`location.locality`	String	❌	—	Locality or area (e.g., "Kothrud")
`location.coordinates`	GeoJSON Point	❌	—	`{ type: "Point", coordinates: [lng, lat] }`
`onboarding_completed`	Boolean	✅	`false`	Whether interests + location are set
`email_verified`	Boolean	✅	`false`	Whether email has been verified
`phone_verified`	Boolean	✅	`false`	Whether phone has been verified
`is_active`	Boolean	✅	`true`	Soft-delete flag
`created_at`	DateTime	✅	Now	Account creation timestamp
`updated_at`	DateTime	✅	Now	Last profile update timestamp

Sample Document

```
json
{
  "_id": { "$oid": "6719a1b2c3d4e5f6a7b8c9d0" },
  "email": "priya.sharma@gmail.com",
  "phone": "9876543210",
  "full_name": "Priya Sharma",
  "password_hash": "$2b$12$LJ3m4k5n6o7p8q9r0s1t2uVwXyZ...",
  "role": "participant",
  "interests": ["hackathons", "tech", "workshops"],
  "location": {
    "city": "Pune",
    "locality": "Kothrud",
    "coordinates": {
      "type": "Point",
      "coordinates": [73.8567, 18.5204]
    }
  },
  "onboarding_completed": true,
  "email_verified": false,
  "phone_verified": false,
  "is_active": true,
  "created_at": { "$date": "2026-10-20T10:30:00Z" },
  "updated_at": { "$date": "2026-10-20T10:35:00Z" }
}
```

Indexes

Index	Fields	Type	Rationale
`idx email_ unique`	`{ email: 1 }`	Unique (collation: `{ locale: "en", strength: 2 }`)	Case-insensitive email uniqueness
`idx phone_ unique`	`{ phone: 1 }`	Unique	Phone uniqueness
`idx locati on_geo`	`{ "location.coordinates": "2dsphere" }`	2dsphere (sparse)	Geospatial proximity queries

9.3 `events` Collection

Field Descriptions

Field	Type	Required	Default	Description
`_id`	ObjectId	Auto	—	MongoDB primary key
`title`	String	✅	—	Event title (max 150 chars)
`description`	String	✅	—	Event description (max 5000 chars)
`category`	String	✅	—	Enum: `"hackathons"`, `"workshops"`, `"summits_conferences"`, `"meetups"`
`tags`	Array[String]	❌	`[]`	Additional topic tags (e.g., `["AI", "web3", "python"]`)
`organizer_id`	ObjectId	✅	—	Reference to users._id
`venue`	Object	✅	—	Embedded venue information
`venue.name`	String	✅	—	Venue name (e.g., "Symbiosis Institute")
`venue.address`	String	✅	—	Street address
`venue.city`	String	✅	—	City name (e.g., "Pune")
`venue.coordinates`	GeoJSON Point	❌	—	`{ type: "Point", coordinates: [lng, lat] }`
`start_date`	DateTime	✅	—	Event start date and time
`end_date`	DateTime	✅	—	Event end date and time
`registration_deadline`	DateTime	✅	—	Last date/time to register
`max_capacity`	Integer	✅	—	Maximum participants (min: 1)
`registered_count`	Integer	✅	`0`	Current confirmed registrations (atomically managed)
`status`	String	✅	`"draft"`	Enum: `"draft"`, `"published"`, `"cancelled"`
`cover_image_url`	String	❌	—	URL to cover image
`contact_email`	String	❌	—	Contact email for the event

Field	Type	Required	Default	Description
`is_free`	Boolean	✅	`true`	Whether the event is free
`created_at`	DateTime	✅	Now	Creation timestamp
`updated_at`	DateTime	✅	Now	Last update timestamp

Sample Document

```
json
{
  "_id": { "$oid": "6719b2c3d4e5f6a7b8c9d0e1" },
  "title": "HackPune 2026 — 36-Hour National Hackathon",
  "description": "Join 500+ developers for a 36-hour coding marathon. Build innovative
solutions in AI, Web3, IoT, or HealthTech. ₹2,00,000 in prizes. Meals and swag included.",
  "category": "hackathons",
  "tags": ["AI", "web3", "IoT", "healthtech", "open-innovation"],
  "organizer_id": { "$oid": "6719a1b2c3d4e5f6a7b8c9d1" },
  "venue": {
    "name": "Symbiosis Institute of Technology",
    "address": "Gram Lavale, Mulshi Taluka",
    "city": "Pune",
    "coordinates": {
      "type": "Point",
      "coordinates": [73.7898, 18.5642]
    }
  },
  "start_date": { "$date": "2026-11-15T09:00:00Z" },
  "end_date": { "$date": "2026-11-16T21:00:00Z" },
  "registration_deadline": { "$date": "2026-11-10T23:59:59Z" },
  "max_capacity": 500,
  "registered_count": 247,
  "status": "published",
  "cover_image_url": "https://example.com/hackpune-banner.jpg",
  "contact_email": "hackpune@symbiosis.edu",
  "is_free": true,
  "created_at": { "$date": "2026-10-01T12:00:00Z" },
  "updated_at": { "$date": "2026-10-15T09:30:00Z" }
}
```

Indexes

Index	Fields	Type	Rationale
`idx_cat_status_date`	`{ category: 1, status: 1, start_date: 1 }`	Compound	Category card queries; filters published events by category sorted by date
`idx_status_date`	`{ status: 1, start_date: 1 }`	Compound	List all published upcoming events
`idx_organizer`	`{ organizer_id: 1 }`	Single	Organizer's own events
`idx_venue_geo`	`{ "venue.coordinates": "2dsphere" }`	2dsphere (sparse)	Geospatial proximity queries
`idx_text_search`	`{ title: "text", description: "text" }`	Text	Keyword search

Why `registered_count` Is Denormalized

Counting registrations by querying the `registrations` collection for every capacity check would be slow and race-condition-prone. Instead:

1. `registered_count` lives on the event document
2. Registration uses `findOneAndUpdate` with atomic `$inc`:

```
python
result = await db.events.find_one_and_update(
    {
        "_id": event_id,
        "status": "published",
        "registered_count": {"$lt": max_capacity},  # capacity check
        "registration_deadline": {"$gt": datetime.utcnow()}  # deadline check
    },
    {"$inc": {"registered_count": 1}},
    return_document=ReturnDocument.AFTER
)
# If result is None → capacity full OR deadline passed OR not published
```

3. Cancellation uses `$inc: {"registered_count": -1}` atomically
4. This guarantees **no overselling** even under concurrent requests

9.4 `registrations` Collection

Field Descriptions

Field	Type	Required	Default	Description
`_id`	ObjectId	Auto	—	MongoDB primary key
`event_id`	ObjectId	✅	—	Reference to events._id
`participant_id`	ObjectId	✅	—	Reference to users._id
`status`	String	✅	`"confirmed"`	Enum: `"confirmed"`, `"cancelled"`
`registered_at`	DateTime	✅	Now	Registration timestamp
`cancelled_at`	DateTime	❌	—	Cancellation timestamp (set on cancel)

Sample Document

```
json
{
  "_id": { "$oid": "6719c3d4e5f6a7b8c9d0e1f2" },
  "event_id": { "$oid": "6719b2c3d4e5f6a7b8c9d0e1" },
  "participant_id": { "$oid": "6719a1b2c3d4e5f6a7b8c9d0" },
  "status": "confirmed",
  "registered_at": { "$date": "2026-10-21T14:22:00Z" },
  "cancelled_at": null
}
```

Indexes

Index	Fields	Type	Rationale
`idx event_participant_unique`	`{ event id: 1, participant_id: 1 }`	**Unique**	Prevents duplicate registrations at DB level
`idx_participant_status`	`{ participant_id: 1, status: 1 }`	Compound	My Events query
`idx_event_status`	`{ event_id: 1, status: 1 }`	Compound	Organizer registration list

Why Registrations Are a Separate Collection

- **Query flexibility:** "My Events" needs registrations by participant; "Organizer view" needs registrations by event. Separate collection supports both efficiently.
- **Cancellation tracking:** Registration status changes are recorded with timestamps.
- **Unique constraint:** The compound unique index on `{event_id, participant_id}` prevents duplicates at the database level, regardless of application logic.
- **Avoiding unbounded arrays:** Embedding registrations in events or users would create unbounded arrays that degrade performance.

9.5 `notifications` Collection

Field Descriptions

Field	Type	Required	Default	Description
`_id`	ObjectId	Auto	—	MongoDB primary key
`user_id`	ObjectId	✅	—	Recipient user
`type`	String	✅	—	Enum: `"registration_confirmed"`, `"registration cancelled"`, `"event_updated"`, `"event_cancelled"`
`title`	String	✅	—	Short notification title
`message`	String	✅	—	Notification message body
`event_id`	ObjectId	❌	—	Related event (for deep-linking)
`is_read`	Boolean	✅	`false`	Read/unread state
`created_at`	DateTime	✅	Now	Creation timestamp

Sample Document

```
json
{
  "_id": { "$oid": "6719d4e5f6a7b8c9d0e1f2a3" },
  "user_id": { "$oid": "6719a1b2c3d4e5f6a7b8c9d0" },
  "type": "registration_confirmed",
  "title": "Registration Confirmed!",
  "message": "You have successfully registered for HackPune 2026 — 36-Hour National
Hackathon on November 15, 2026.",
  "event_id": { "$oid": "6719b2c3d4e5f6a7b8c9d0e1" },
  "is_read": false,
  "created_at": { "$date": "2026-10-21T14:22:01Z" }
}
```

Indexes

Index	Fields	Type	Rationale
`idx user_read_date`	`{ user id: 1, is read: 1, created_at: -1 }`	Compound	Fetch unread notifications newest-first
`idx_user_date`	`{ user id: 1, created_at: -1 }`	Compound	All notifications for a user
`idx_ttl`	`{ created_at: 1 }`	TTL (expireAfterSeconds: 7776000)	Auto-delete after 90 days

9.6 Concurrency Considerations

Scenario	Risk	Solution
Two participants register for the last seat simultaneously	Both might succeed → overselling	`findOneAndUpdate` with `registered_count < max_capacity` condition; only one write wins
Registration + capacity increment not atomic	Count mismatch	Single `findOneAndUpdate` is atomic; registration insert in same logical flow
Cancellation decrement below zero	Negative count	Only decrement if active registration exists; use `findOneAndUpdate` with `status: "confirmed"`
Concurrent profile updates	Last-write-wins	Acceptable for profile data; no critical business rule
Notification bulk insert for 100 registrants	Slow sequential inserts	Use `insert_many` for batch notification creation

10. Complete API Contract

10.1 Authentication

`POST /api/auth/signup`

Create a new user account.

Aspect	Details
Auth	None
Roles	None (public)

Request:

```
json
{
  "full_name": "Priya Sharma",
  "email": "priya.sharma@gmail.com",
  "phone": "9876543210",
  "password": "SecureP@ss123",
  "role": "participant"
}
```

Success Response (201):

```
json
{
  "user": {
    "id": "6719a1b2c3d4e5f6a7b8c9d0",
    "full_name": "Priya Sharma",
    "email": "priya.sharma@gmail.com",
    "phone": "9876543210",
    "role": "participant",
    "onboarding_completed": false,
    "created_at": "2026-10-20T10:30:00Z"
  },
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

Error Responses:

Status	Condition
409	Email already registered
409	Phone number already registered
422	Missing required fields or invalid format

`POST /api/auth/login`

Authenticate and receive a JWT.

Aspect	Details
Auth	None

Request:

```
json
{
  "email": "priya.sharma@gmail.com",
  "password": "SecureP@ss123"
}
```

Success Response (200):

```
json
{
  "user": {
    "id": "6719a1b2c3d4e5f6a7b8c9d0",
    "full_name": "Priya Sharma",
    "email": "priya.sharma@gmail.com",
    "role": "participant",
    "onboarding_completed": true
  },
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

Error Responses:

Status	Condition
401	Invalid email or password
422	Missing fields

`GET /api/auth/me`

Get the current authenticated user's profile.

Aspect	Details
Auth	✅ Bearer token required

Success Response (200):

```
json
{
  "id": "6719a1b2c3d4e5f6a7b8c9d0",
  "full_name": "Priya Sharma",
  "email": "priya.sharma@gmail.com",
  "phone": "9876543210",
  "role": "participant",
  "interests": ["hackathons", "tech", "workshops"],
  "location": {
    "city": "Pune",
    "locality": "Kothrud"
  },
  "onboarding_completed": true,
  "created_at": "2026-10-20T10:30:00Z"
}
```

10.2 Onboarding

`PUT /api/onboarding/interests`

Set or update participant interests.

Aspect	Details
Auth	✅ Participant or Both

Request:

```
json
{
  "interests": ["hackathons", "tech", "workshops"]
}
```

Valid interests: `"hackathons"`, `"workshops"`, `"summits_conferences"`, `"meetups"`, `"tech"`, `"design"`, `"business"`, `"health"`, `"arts"`, `"science"`, `"education"`, `"sports"`

Success Response (200):

```
json
{
  "message": "Interests updated successfully",
  "interests": ["hackathons", "tech", "workshops"]
}
```

Status	Condition
400	Empty interests array
422	Invalid interest value

`PUT /api/onboarding/location`

Set or update preferred location.

Aspect	Details
Auth	✅ Participant or Both

Request:

```
json
{
  "city": "Pune",
  "locality": "Kothrud",
  "coordinates": {
    "type": "Point",
    "coordinates": [73.8567, 18.5204]
  }
}
```

`locality` and `coordinates` are optional. `city` is required.

Success Response (200):

```
json
{
  "message": "Location updated successfully",
  "location": {
    "city": "Pune",
    "locality": "Kothrud"
  }
}
```

`POST /api/onboarding/complete`

Mark onboarding as completed. Validates that interests and location are set.

Aspect	Details
Auth	✅ Participant or Both

Success Response (200):

```
json
{
  "message": "Onboarding completed",
  "onboarding_completed": true
}
```

Status	Condition
400	Interests empty or location not set

10.3 Profile

`GET /api/profile`

Get full user profile.

Aspect	Details
Auth	✅ Any authenticated user

Response: Same as `GET /api/auth/me`

`PUT /api/profile`

Update profile fields.

Aspect	Details
Auth	✅ Any authenticated user

Request (all fields optional):

```
json
{
  "full_name": "Priya S. Sharma",
  "phone": "9876543211",
  "interests": ["hackathons", "tech", "design"],
  "location": {
    "city": "Mumbai",
    "locality": "Andheri"
  }
}
```

Success Response (200):

```
json
{
  "message": "Profile updated successfully",
  "user": { "...updated user object..." }
}
```

Status	Condition
409	Phone number already taken by another user
422	Invalid field values

`PUT /api/profile/password`

Change password.

Aspect	Details
Auth	✅ Any authenticated user

Request:

```
json
{
  "current_password": "SecureP@ss123",
  "new_password": "NewSecureP@ss456"
}
```

Success Response (200):

```
json
{
  "message": "Password changed successfully"
}
```

Status	Condition
400	Current password is incorrect
422	New password too weak

10.4 Events (Public + Participant)

`GET /api/events`

List published events with optional filters.

Aspect	Details
Auth	❌ Public

Query Parameters:

Parameter	Type	Description
`category`	string	Filter by category
`city`	string	Filter by venue city
`search`	string	Text search in title/description
`date_from`	ISO date	Events starting after this date
`date_to`	ISO date	Events starting before this date
`page`	int	Page number (default: 1)
`limit`	int	Results per page (default: 20, max: 50)

Success Response (200):

```
json
{
  "events": [
    {
      "id": "6719b2c3d4e5f6a7b8c9d0e1",
      "title": "HackPune 2026 — 36-Hour National Hackathon",
      "description": "Join 500+ developers for a 36-hour coding marathon...",
      "category": "hackathons",
      "tags": ["AI", "web3", "IoT"],
      "venue": {
        "name": "Symbiosis Institute of Technology",
        "city": "Pune"
      },
      "start_date": "2026-11-15T09:00:00Z",
      "end_date": "2026-11-16T21:00:00Z",
      "registration_deadline": "2026-11-10T23:59:59Z",
      "max_capacity": 500,
      "registered_count": 247,
      "is_free": true,
      "cover_image_url": "https://example.com/hackpune-banner.jpg",
      "organizer_name": "Rahul Mehta"
    }
  ],
  "total": 42,
  "page": 1,
  "limit": 20
}
```

`GET /api/events/categories/{category}`

This is the endpoint that powers category card drill-downs. When a user clicks the "Hackathons", "Workshops", "Summits & Conferences", or "Meetups" card on the dashboard, the frontend calls this endpoint to fetch all published events in that category.

Aspect	Details
Auth	❌ Public
Path param	`category` : one of `hackathons` , `workshops` , `summits_conferences` , `meetups`

Query Parameters:

Parameter	Type	Description
`city`	string	Filter by venue city
`page`	int	Page number (default: 1)
`limit`	int	Results per page (default: 20)
`sort`	string	`"date_asc"`, `"date_desc"` (default), `"popularity"`

Success Response (200):

```
json
{
  "category": "hackathons",
  "category_display": "Hackathons",
  "events": [
    {
      "id": "6719b2c3d4e5f6a7b8c9d0e1",
      "title": "HackPune 2026 — 36-Hour National Hackathon",
      "description": "Join 500+ developers for a 36-hour coding marathon...",
      "category": "hackathons",
      "tags": ["AI", "web3", "IoT"],
      "venue": {
        "name": "Symbiosis Institute of Technology",
        "address": "Gram Lavale, Mulshi Taluka",
        "city": "Pune"
      },
      "start_date": "2026-11-15T09:00:00Z",
      "end_date": "2026-11-16T21:00:00Z",
      "registration_deadline": "2026-11-10T23:59:59Z",
      "max_capacity": 500,
      "registered_count": 247,
      "spots_remaining": 253,
      "registration_open": true,
      "is_free": true,
      "cover_image_url": "https://example.com/hackpune-banner.jpg",
      "organizer_name": "Rahul Mehta"
    },
    {
      "id": "6719b2c3d4e5f6a7b8c9d0e5",
      "title": "BuildWithAI Hackathon — Bangalore Edition",
      "description": "24-hour AI-focused hackathon. Use Google Gemini, OpenAI, or any AI
tool...",
      "category": "hackathons",
      "tags": ["AI", "machine-learning", "LLM"],
      "venue": {
        "name": "91springboard",
        "address": "Koramangala, 4th Block",
        "city": "Bangalore"
      },
      "start_date": "2026-11-22T10:00:00Z",
      "end_date": "2026-11-23T10:00:00Z",
      "registration_deadline": "2026-11-18T23:59:59Z",
      "max_capacity": 200,
      "registered_count": 89,
      "spots_remaining": 111,
```

```
      "registration_open": true,
      "is_free": true,
      "cover_image_url": null,
      "organizer_name": "AI Club Bangalore"
    }
  ],
  "total": 8,
  "page": 1,
  "limit": 20
}
```

Each event card in the response includes:

- Title, description, category, tags
- Venue details with city
- Date range and registration deadline
- `spots_remaining` (computed: `max_capacity - registered_count`)
- `registration_open` (computed: `registration_deadline > now AND registered_count < max_capacity AND status == "published"`)
- Cover image URL and organizer name

Status	Condition
422	Invalid category value

`GET /api/events/{event_id}`

Get full details of a single event.

Aspect	Details
Auth	❌ Public (published) / ✅ Organizer (own drafts)

Success Response (200):

```
json
{
  "id": "6719b2c3d4e5f6a7b8c9d0e1",
  "title": "HackPune 2026 — 36-Hour National Hackathon",
  "description": "Join 500+ developers for a 36-hour coding marathon. Build innovative
solutions in AI, Web3, IoT, or HealthTech. ₹2,00,000 in prizes. Meals and swag included.",
  "category": "hackathons",
  "tags": ["AI", "web3", "IoT", "healthtech", "open-innovation"],
  "venue": {
    "name": "Symbiosis Institute of Technology",
    "address": "Gram Lavale, Mulshi Taluka",
    "city": "Pune",
    "coordinates": { "type": "Point", "coordinates": [73.7898, 18.5642] }
  },
  "start_date": "2026-11-15T09:00:00Z",
  "end_date": "2026-11-16T21:00:00Z",
  "registration_deadline": "2026-11-10T23:59:59Z",
  "max_capacity": 500,
  "registered_count": 247,
  "spots_remaining": 253,
  "registration_open": true,
  "status": "published",
  "is_free": true,
  "cover_image_url": "https://example.com/hackpune-banner.jpg",
  "contact_email": "hackpune@symbiosis.edu",
  "organizer": {
    "id": "6719a1b2c3d4e5f6a7b8c9d1",
    "full_name": "Rahul Mehta"
  },
  "is_registered": false,
  "created_at": "2026-10-01T12:00:00Z"
}
```

`is_registered` is `true` if the authenticated user has an active registration; `null` if not authenticated.

`GET /api/events/recommendations`

Get personalized event recommendations for the authenticated participant.

Aspect	Details
Auth	✅ Participant or Both (onboarding completed)

Query Parameters:

Parameter	Type	Description
`limit`	int	Max recommendations (default: 10)

Success Response (200):

```
json
{
  "recommendations": [
    {
      "id": "6719b2c3d4e5f6a7b8c9d0e1",
      "title": "HackPune 2026 — 36-Hour National Hackathon",
      "category": "hackathons",
      "venue": { "name": "Symbiosis Institute of Technology", "city": "Pune" },
      "start_date": "2026-11-15T09:00:00Z",
      "registration_deadline": "2026-11-10T23:59:59Z",
      "max_capacity": 500,
      "registered_count": 247,
      "spots_remaining": 253,
      "registration_open": true,
      "is_free": true,
      "cover_image_url": "https://example.com/hackpune-banner.jpg",
      "relevance_score": 0.87,
      "match_reasons": ["Matches your interest in hackathons", "Near your location in
Pune"]
    }
  ]
}
```

10.5 Organizer Event Management

`POST /api/organizer/events`

Create a new event.

Aspect	Details
Auth	✅ Organizer or Both

Request:

```
json
{
  "title": "Python Workshop — FastAPI Masterclass",
  "description": "Learn to build production-ready APIs with FastAPI. Covers async
programming, Pydantic validation, authentication, and MongoDB integration.",
  "category": "workshops",
  "tags": ["python", "fastapi", "backend"],
  "venue": {
    "name": "WeWork Galaxy",
    "address": "Residency Road",
    "city": "Bangalore"
  },
  "start_date": "2026-11-20T10:00:00Z",
  "end_date": "2026-11-20T17:00:00Z",
  "registration_deadline": "2026-11-18T23:59:59Z",
  "max_capacity": 50,
  "status": "draft",
  "cover_image_url": "https://example.com/fastapi-workshop.jpg",
  "contact_email": "workshops@devclub.in",
  "is_free": true
}
```

Success Response (201):

```
json
{
  "message": "Event created successfully",
  "event": {
    "id": "6719e5f6a7b8c9d0e1f2a3b4",
    "title": "Python Workshop — FastAPI Masterclass",
    "status": "draft",
    "organizer_id": "6719a1b2c3d4e5f6a7b8c9d1",
    "created_at": "2026-10-22T11:00:00Z"
  }
}
```

Status	Condition
403	User is not an organizer
422	Missing required fields, invalid dates, end_date before start_date, deadline after start_date

`GET /api/organizer/events`

List the organizer's own events (all statuses).

Aspect	Details
Auth	✅ Organizer or Both

Success Response (200):

```
json
{
  "events": [
    {
      "id": "6719e5f6a7b8c9d0e1f2a3b4",
      "title": "Python Workshop — FastAPI Masterclass",
      "category": "workshops",
      "status": "draft",
      "start_date": "2026-11-20T10:00:00Z",
      "registered_count": 0,
      "max_capacity": 50,
      "created_at": "2026-10-22T11:00:00Z"
    }
  ],
  "total": 3
}
```

`GET /api/organizer/events/{event_id}`

Get organizer's event with full details and stats.

Aspect	Details
Auth	✅ Organizer (owner only)

Status	Condition
403	Not the event owner
404	Event not found

`PUT /api/organizer/events/{event_id}`

Update event details. If the event is published and date/time/venue changes, notifications are created for registrants.

Aspect	Details
Auth	✅ Organizer (owner only)

Request (all fields optional):

```
json
{
  "title": "Python Workshop – FastAPI Masterclass (Updated Venue)",
  "venue": {
    "name": "91springboard",
    "address": "HSR Layout",
    "city": "Bangalore"
  },
  "start_date": "2026-11-21T10:00:00Z"
}
```

Success Response (200):

```
json
{
  "message": "Event updated successfully",
  "event": { "...updated event..." },
  "notifications_sent": 15
}
```

`notifications_sent` indicates how many registrant notifications were created (only for important changes to published events).

`PATCH /api/organizer/events/{event_id}/publish`

Publish a draft event.

Aspect	Details
Auth	✅ Organizer (owner only)

Success Response (200):

```
json
{
  "message": "Event published successfully",
  "event": { "...event with status: published..." }
}
```

Status	Condition
400	Event is not in draft status
403	Not the owner

`PATCH /api/organizer/events/{event_id}/cancel`

Cancel an event. Creates notifications for all registrants.

Aspect	Details
Auth	✅ Organizer (owner only)

Success Response (200):

```
json
{
  "message": "Event cancelled successfully",
  "notifications_sent": 47
}
```

Status	Condition
400	Already cancelled
403	Not the owner

`GET /api/organizer/events/{event_id}/registrations`

List registered participants for the organizer's event.

Aspect	Details
Auth	✅ Organizer (owner only)

Success Response (200):

```
json
{
  "event_id": "6719b2c3d4e5f6a7b8c9d0e1",
  "event_title": "HackPune 2026",
  "total_registered": 247,
  "registrations": [
    {
      "participant": {
        "full_name": "Priya Sharma",
        "email": "priya.sharma@gmail.com",
        "phone": "9876543210"
      },
      "registered_at": "2026-10-21T14:22:00Z",
      "status": "confirmed"
    }
  ],
  "page": 1,
  "limit": 50
}
```

10.6 Registrations

`POST /api/registrations/{event_id}`

Register for an event.

Aspect	Details
Auth	✅ Participant or Both

Success Response (201):

```
json
{
  "message": "Successfully registered for the event",
  "registration": {
    "id": "6719c3d4e5f6a7b8c9d0e1f2",
    "event_id": "6719b2c3d4e5f6a7b8c9d0e1",
    "event_title": "HackPune 2026 — 36-Hour National Hackathon",
    "status": "confirmed",
    "registered_at": "2026-10-21T14:22:00Z"
  }
}
```

Status	Condition
400	Registration deadline passed
400	Event capacity full
400	Event is not published
400	Event is cancelled
401	Not authenticated
403	Not a participant
409	Already registered for this event

`DELETE /api/registrations/{event_id}`

Cancel a registration.

Aspect	Details
Auth	✅ Participant or Both (own registration)

Success Response (200):

```
json
{
  "message": "Registration cancelled successfully",
  "event_id": "6719b2c3d4e5f6a7b8c9d0e1"
}
```

Status	Condition
400	Event already started
400	Registration already cancelled
404	No registration found for this event

`GET /api/registrations/my-events`

Get participant's registered events.

Aspect	Details
Auth	✅ Participant or Both

Query Parameters:

Parameter	Type	Description
`status`	string	`"upcoming"`, `"past"`, `"cancelled"` (default: all)
`page`	int	Page number
`limit`	int	Results per page

Success Response (200):

```
json
{
  "registrations": [
    {
      "registration_id": "6719c3d4e5f6a7b8c9d0e1f2",
      "registration_status": "confirmed",
      "registered_at": "2026-10-21T14:22:00Z",
      "event": {
        "id": "6719b2c3d4e5f6a7b8c9d0e1",
        "title": "HackPune 2026 — 36-Hour National Hackathon",
        "category": "hackathons",
        "venue": { "name": "Symbiosis Institute of Technology", "city": "Pune" },
        "start_date": "2026-11-15T09:00:00Z",
        "end_date": "2026-11-16T21:00:00Z",
        "status": "published"
      }
    }
  ],
  "total": 3,
  "page": 1,
  "limit": 20
}
```

10.7 Notifications

`GET /api/notifications`

Get user's notifications (newest first).

Aspect	Details
Auth	✅ Any authenticated user

Query Parameters:

Parameter	Type	Description
`unread_only`	bool	Only unread notifications
`page`	int	Page number
`limit`	int	Results per page (default: 20)

Success Response (200):

```
json
{
  "notifications": [
    {
      "id": "6719d4e5f6a7b8c9d0e1f2a3",
      "type": "registration_confirmed",
      "title": "Registration Confirmed!",
      "message": "You have successfully registered for HackPune 2026.",
      "event_id": "6719b2c3d4e5f6a7b8c9d0e1",
      "is_read": false,
      "created_at": "2026-10-21T14:22:01Z"
    }
  ],
  "total": 5,
  "unread_count": 2,
  "page": 1,
  "limit": 20
}
```

`GET /api/notifications/unread-count`

Aspect	Details
Auth	✅ Any authenticated user

Success Response (200):

```
json
{
  "unread_count": 3
}
```

`PATCH /api/notifications/{notification_id}/read`

Mark a single notification as read.

Aspect	Details
Auth	✅ Any authenticated user (own notifications only)

Success Response (200):

```
json
{
  "message": "Notification marked as read"
}
```

`PATCH /api/notifications/read-all`

Mark all notifications as read.

Aspect	Details
Auth	✅ Any authenticated user

Success Response (200):

```
json
{
  "message": "All notifications marked as read",
  "updated_count": 5
}
```